

Simple Linear Regression – Part-1

Objective

To understand and implement Simple Linear Regression using Python and Machine Learning.

Mathematical Model

$$y = mx + c$$

- **x**: Independent variable
- **y**: Dependent variable
- **m**: Slope
- **c**: Intercept

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

Step 1: Create Dataset

We create a small dataset representing study hours and exam scores.

```
# Create a simple dataset containing study hours and corresponding scores
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8], # Independent variable: number of hours studied
    "Score": [35, 40, 50, 55, 65, 70, 75, 85] # Dependent variable: score obtained
}
```

```
df = pd.DataFrame(data)
```

```
# Display the dataset- first 5 rows
df.head()
```

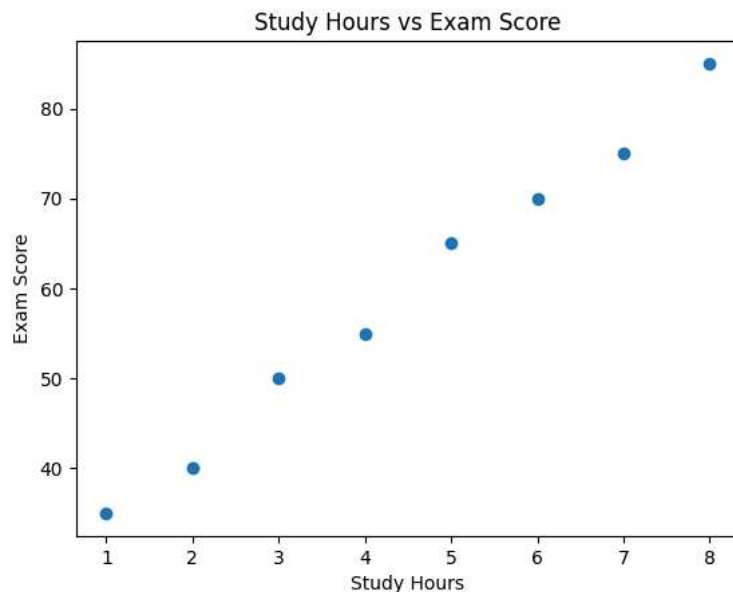
	Hours	Score
0	1	35
1	2	40
2	3	50
3	4	55
4	5	65

Step 2: Define Independent and Dependent Variables

```
# Independent and dependent variables
X = df[["Hours"]]
y = df["Score"]
```

Step 3: Visualize the Dataset

```
plt.scatter(df['Hours'], df['Score'])
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Study Hours vs Exam Score')
plt.show()
```



Step 4: Train the Linear Regression Model

```
model = LinearRegression()
model.fit(X, y)
```

LinearRegression ⓘ ?

```
LinearRegression()
```

Step 5: Predict Using Trained Model

```
y_pred = model.predict(X)
```

Step 6: Display Model Parameters

```
print('Slope (m):', model.coef_[0])
print('Intercept (c):', model.intercept_)
```

```
Slope (m): 7.083333333333334
Intercept (c): 27.499999999999996
```

Step 7: Testing with New Input

```
test_hours = [[9]]
predicted_score = model.predict(test_hours)

print('Study Hours:', test_hours[0][0])
print('Predicted Exam Score:', predicted_score[0])
```

```
Study Hours: 9
Predicted Exam Score: 91.25
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

```
# Take study hours as input from the user
hours = float(input("Enter number of study hours: "))

# Create input in 2D list format as required by the model
test_hours = [[hours]]
```

```
# Predict the exam score using the trained model
predicted_score = model.predict(test_hours)

# Display the input and predicted output
print("Study Hours:", hours)
print("Predicted Exam Score:", predicted_score[0])
```

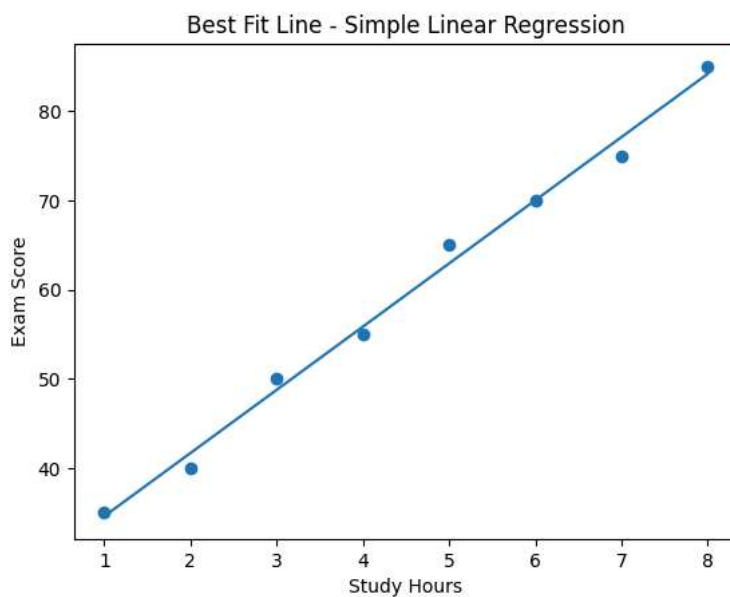
```
Enter number of study hours: 5
Study Hours: 5.0
Predicted Exam Score: 62.91666666666667
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn()
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Step 8: Best Fit Line Visualization

```
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Best Fit Line - Simple Linear Regression')
plt.show()
```



Conclusion

- Simple Linear Regression models the relationship between one input and one output.
- The model learns the best-fit line using training data.
- It can be used to predict future values.

Simple Linear Regression on CSV Dataset

Objective

To implement Simple Linear Regression to predict Placement Package (LPA) based on CGPA.

Part 1: Train and Save the Model

Step 1: Import Required Libraries

```
# Import pandas for data handling
import pandas as pd

# Import matplotlib for data visualization
import matplotlib.pyplot as plt

# Import Linear Regression model
from sklearn.linear_model import LinearRegression
```

Step 2: Load Dataset from CSV File

```
# Read the dataset from CSV file
df = pd.read_csv("/content/drive/MyDrive/ML_EXP_3/Placement.csv")
```

```
# Display first 5 rows of the dataset
df.head()
```

	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57

Step 3: Understand the Dataset

```
# Display the shape of the dataset (rows, columns)
df.shape
```

```
(200, 2)
```

```
# Display dataset information such as column names and data types
df.info()
```

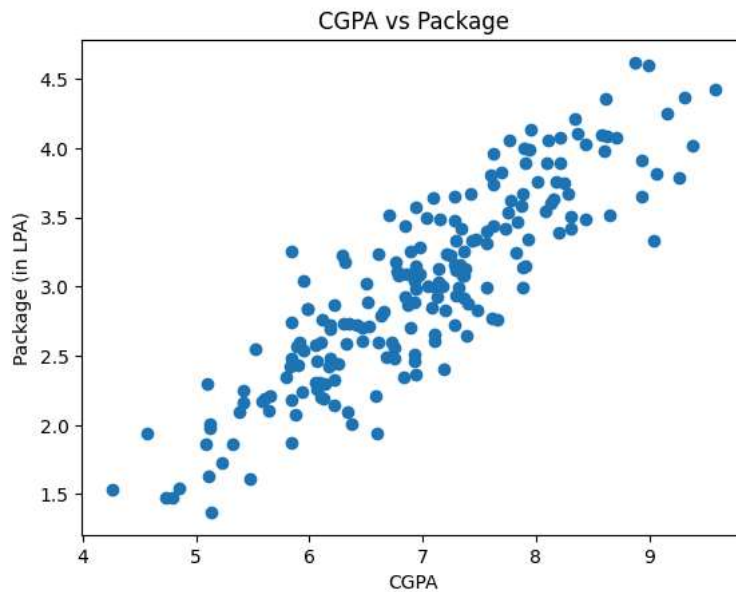
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    cgpa        200 non-null     float64
1    package     200 non-null     float64
dtypes: float64(2)
memory usage: 3.3 KB
```

```
# Display statistical summary of the dataset
df.describe()
```

	cgpa	package
count	200.000000	200.000000
mean	6.990500	2.996050
std	1.069409	0.691644
min	4.260000	1.370000
25%	6.190000	2.487500
50%	6.965000	2.995000
75%	7.737500	3.492500
max	9.580000	4.620000

Step 4: Visualize the Dataset (Scatter Plot)

```
# Create scatter plot between CGPA and Package
plt.scatter(df['cgpa'], df['package'])
plt.xlabel('CGPA')
plt.ylabel('Package (in LPA)')
plt.title('CGPA vs Package')
plt.show()
#Each dot represents one student.
```



Step 5: Define Independent and Dependent Variables

```
# Independent variable (input feature)
X = df[['cgpa']]

# Dependent variable (target)
y = df['package']
```

Step 6: Split Data into Training and Testing Sets

```
# Import train_test_split function
from sklearn.model_selection import train_test_split

# Split the data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2
)
```

Step 7: Create and Train Linear Regression Model

```
# Create Linear Regression model object
model = LinearRegression()

# Train the model using training data
model.fit(X_train, y_train)
```

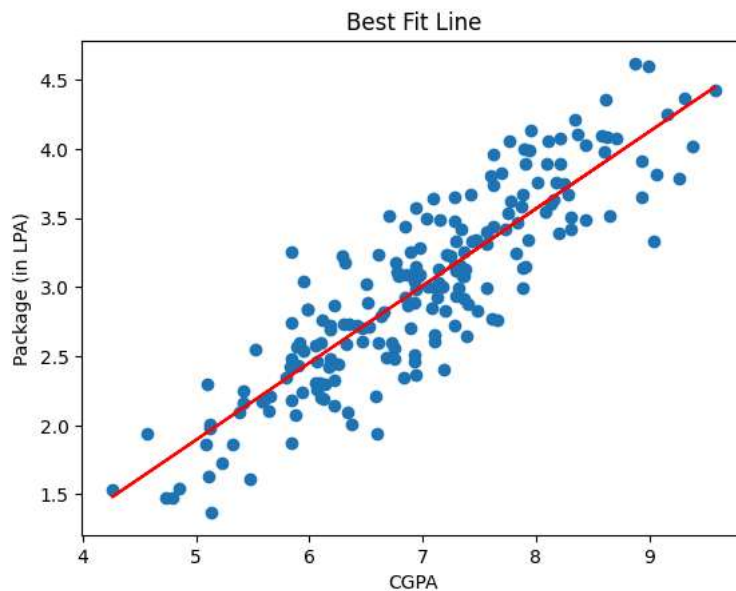
LinearRegression ⓘ ?

Step 8: Plot Best Fit Line

```
# Plot original data
plt.scatter(df['cgpa'], df['package'])

# Plot regression (best fit) line
plt.plot(X_train, model.predict(X_train), color='red')

plt.xlabel('CGPA')
plt.ylabel('Package (in LPA)')
plt.title('Best Fit Line')
plt.show()
```



Step 9: Make Prediction Using Test Data

```
# Predict package for the first test sample
model.predict(X_test.iloc[0].values.reshape(1,1))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
array([3.89111601])
```

Prediction Using User Input

```
# Take CGPA value from the user
cgpa = float(input("Enter CGPA: "))

# Convert input into 2D array format required by the model
cgpa_input = [[cgpa]]

# Predict the placement package using the trained model
predicted_package = model.predict(cgpa_input)

# Display the prediction
print("Entered CGPA:", cgpa)
print("Predicted Package (in LPA):", predicted_package[0])
```

```
Enter CGPA: 9
Entered CGPA: 9.0
Predicted Package (in LPA): 4.125455838582734
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

Step 10: Understand Linear Regression Equation

Simple Linear Regression formula: $y=mx+c$

```
# Get the slope (m) of the regression line
m = model.coef_[0]

# Get the intercept (c) of the regression line
c = model.intercept_

# Print the slope and intercept values
print("Slope (m):", m)
print("Intercept (c):", c)
```

```
Slope (m): 0.557951973425072
Intercept (c): -0.8961119222429144
```

Save the Model

```
import pickle
```

```
pickle.dump(model,open('SLRModel.pkl','wb'))
```

Part 2: Create Flask Backend

```
from flask import Flask,render_template,request
import pickle
import numpy as np

app=Flask(__name__)

model=pickle.load(open("SLRModel.pkl","rb"))

@app.route('/')
def home():
    return render_template("index.html")

@app.route('/predict',methods=['POST'])

def predict():

    cgpa=float(request.form['cgpa'])

    prediction=model.predict(np.array([[cgpa]]))

    return render_template(
        "index.html",
        prediction_text=f"Predicted Package : {prediction[0]:.2f} LPA"
    )

if __name__=="__main__":
    app.run(debug=True)
```

```
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inst
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Part 3: Create the HTML Frontend

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Package Predictor</title>
```

```
<style>

*{
  margin:0;
  padding:0;
  box-sizing:border-box;
  font-family:Arial, Helvetica, sans-serif;
}

body{
  background:#f4f7fc;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}

.container{

  width:420px;
  background:white;
  padding:35px;
  border-radius:15px;
  box-shadow:0px 8px 20px rgba(0,0,0,0.2);
  text-align:center;

}

h1{

  color:#0d47a1;
  margin-bottom:10px;

}

h3{

  color:#555;
  margin-bottom:25px;

}

label{

  display:block;
  text-align:left;
  font-weight:bold;
  margin-bottom:8px;

}

input{

  width:100%;
  padding:12px;
  font-size:16px;
  border:1px solid #ccc;
  border-radius:8px;
  margin-bottom:20px;

}

button{

  width:100%;
  padding:12px;
  background:#1565c0;
  color:white;
  border:none;
  border-radius:8px;
  font-size:18px;
  cursor:pointer;
  transition:0.3s;

}

button:hover{
```



```
        background:#0d47a1;

    }

    .result{

        margin-top:25px;
        padding:15px;
        border-radius:8px;
        background:#e8f5e9;
        color:green;
        font-size:20px;
        font-weight:bold;

    }

    footer{

        margin-top:20px;
        font-size:13px;
        color:#777;

    }

</style>

</head>

<body>

<div class="container">

    <h1>Student Package Predictor</h1>

    <h3>Simple Linear Regression</h3>

    <form action="/predict" method="POST">

        <label>Enter Student CGPA</label>

        <input
            type="number"
            name="cgpa"
            step="0.01"
            min="0"
            max="10"
            placeholder="Example : 8.45"
            required>

        <button type="submit">
            Predict Package
        </button>

    </form>

    {% if prediction_text %}
    <div class="result">
        {{ prediction_text }}
    </div>
    {% endif %}

    <footer>

        Machine Learning by Dr. Sachin Balawant Takmare

    </footer>

</div>

</body>
</html>
```

